

Part 1: Exploring and Applying Ensemble Methods

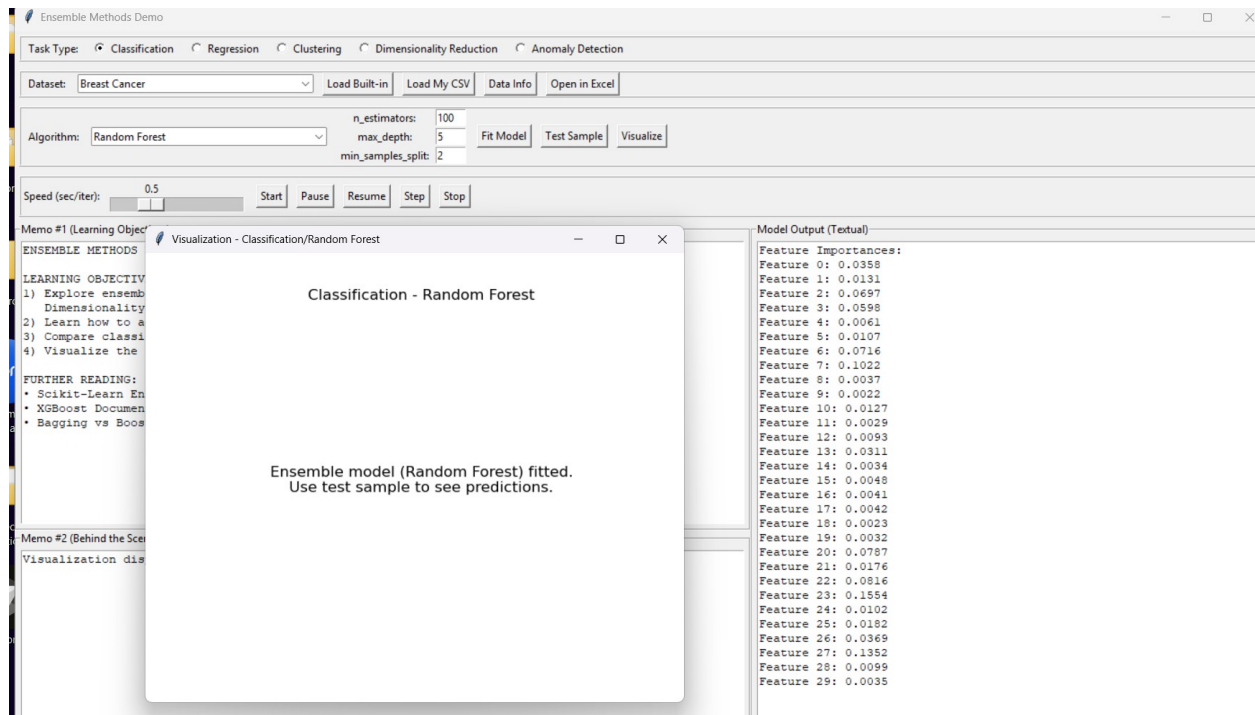


Figure explanation:

The alpha-phase GUI successfully loads the Breast Cancer dataset and fits a Random Forest classification model. The textual output panel shows feature importances, which is useful for interpretation. However, the visualization window does not present meaningful classification performance plots such as a confusion matrix, decision boundary, ROC curve, or accuracy summary. Instead, it only displays a static text message, which limits the usefulness of the visualization component.

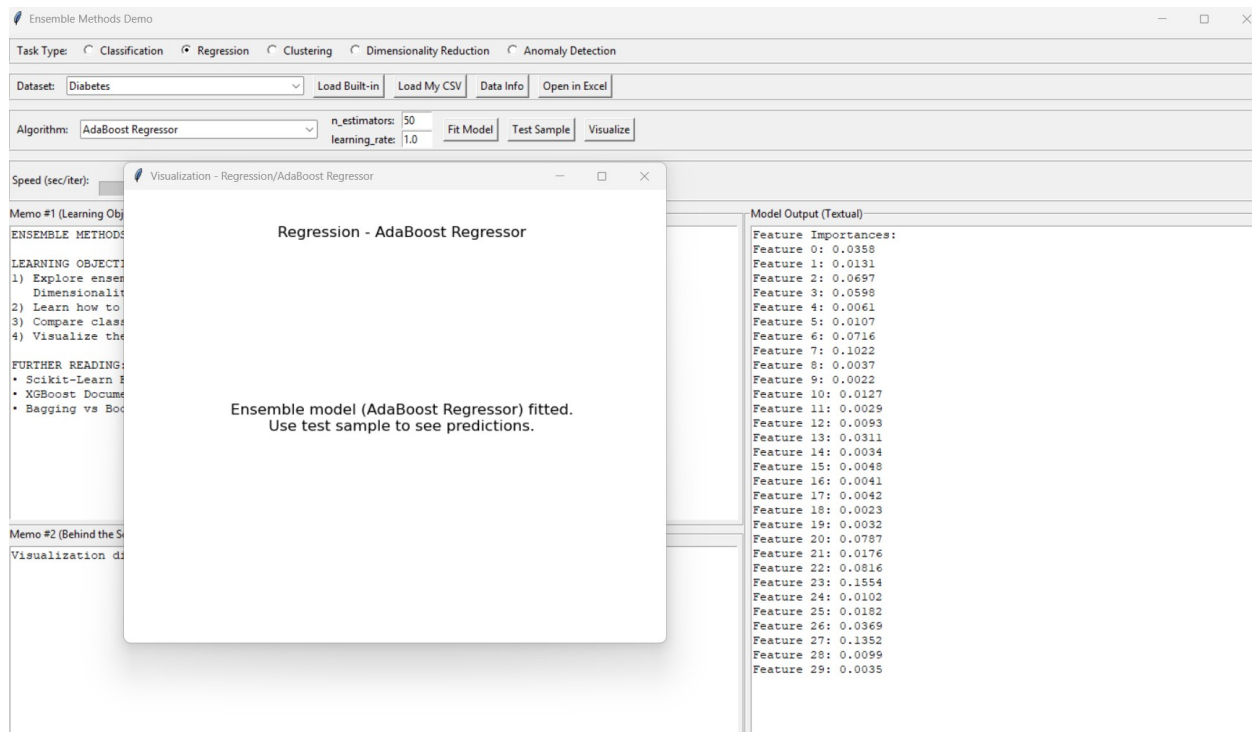
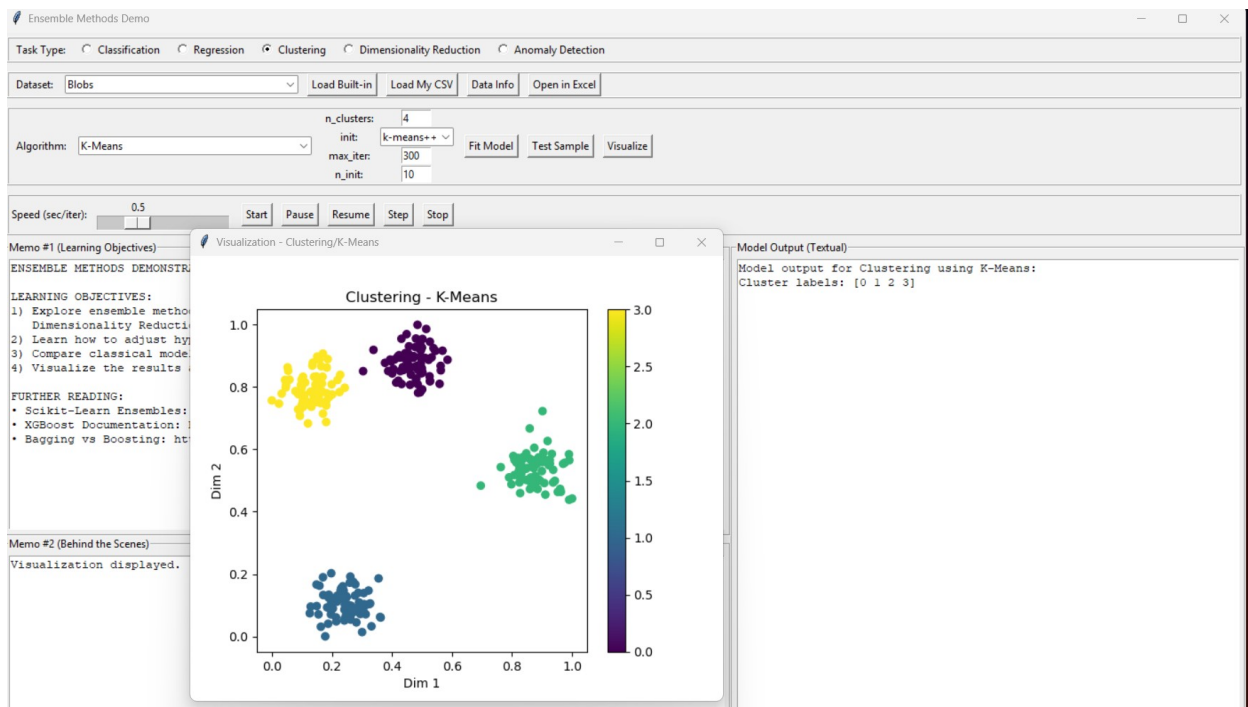


Figure Explanation (Regression)

The regression task was tested using the Diabetes dataset with the AdaBoost Regressor model. The system successfully fitted the model and displayed feature importances, indicating that the model training process works correctly. However, the visualization output is limited, as it only displays a static text message instead of meaningful regression plots such as predicted vs actual values, residual plots, or evaluation metrics like RMSE or R^2 . This limits the usefulness of the visualization for understanding model performance.



clustering:

The clustering task using K-Means on the Blobs dataset produced a meaningful visualization, displaying clusters in a two-dimensional space. Unlike classification and regression tasks, clustering visualization is more effective because it directly represents data grouping. This indicates that the visualization component works better for unsupervised learning tasks.

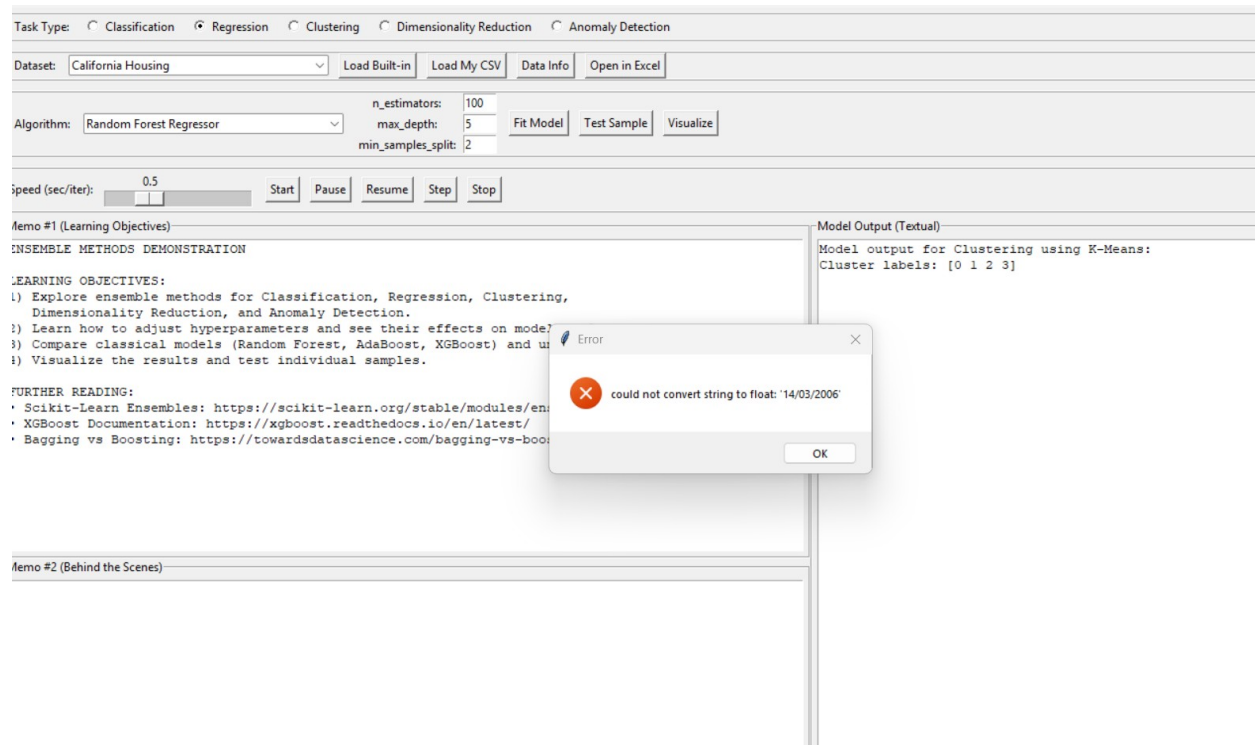


Figure explanation:

When the custom CSV file `Aquifer_Petrignano.csv` was loaded, the alpha-phase program produced an error stating that it could not convert the date value `'14/03/2006'` to float. This indicates that the CSV loading pipeline is not able to handle non-numeric columns such as date fields. The program attempts to scale the full dataset directly as numeric input, which causes failure for real-world datasets containing timestamps or categorical information. This highlights an important usability and preprocessing limitation in the current implementation.

Additional issue discovered during CSV testing

A further problem was identified when testing the custom aquifer dataset. The program crashed because it attempted to convert the Date column directly into a floating-point value. This shows that the CSV loading procedure does not include basic preprocessing steps for date or non-numeric variables. In practical machine learning workflows, date fields should either be transformed into engineered numeric features or excluded before scaling and fitting. The lack of such preprocessing makes the current CSV loading functionality unsuitable for many real datasets.

Part 2: Evaluating and Validating Your Model

```
]: results = [
    evaluate_model("Baseline Linear Regression", y_test, y_pred_baseline, X_test.shape[1]),
    evaluate_model("Random Forest", y_test, y_pred_rf, X_test.shape[1]),
    evaluate_model("AdaBoost", y_test, y_pred_ada, X_test.shape[1]),
    evaluate_model("XGBoost", y_test, y_pred_xgb, X_test.shape[1]),
]

results_df = pd.DataFrame(results)
results_df.sort_values(by="R2", ascending=False)
```

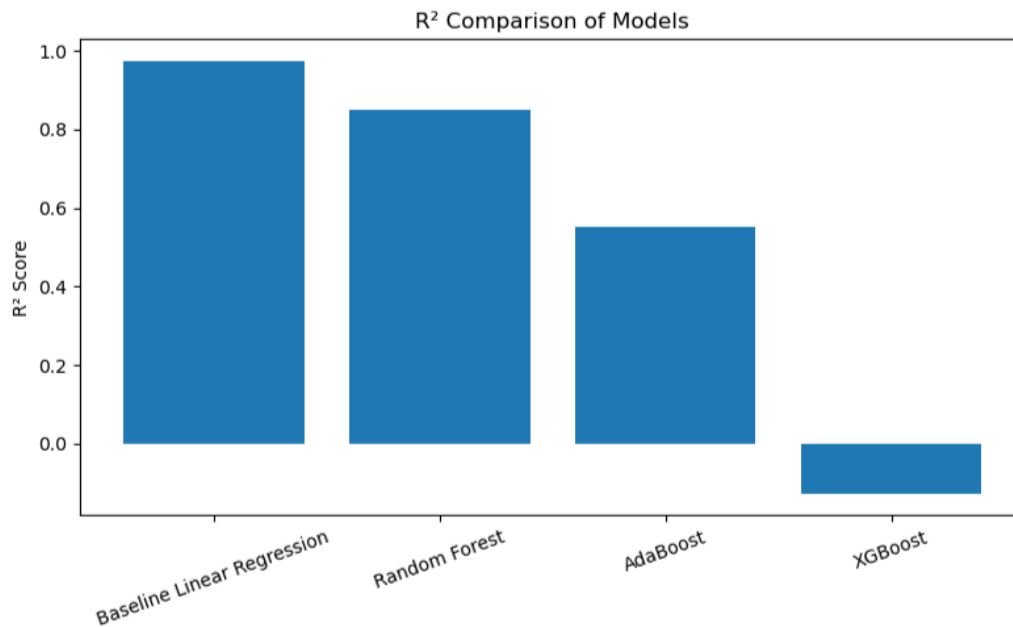
```
]:
```

	Model	MSE	MAE	RMSE	R2	Adjusted_R2
0	Baseline Linear Regression	0.027985	0.119233	0.167286	0.974159	0.973907
1	Random Forest	0.161768	0.269445	0.402204	0.850626	0.849165
2	AdaBoost	0.487008	0.577385	0.697859	0.550305	0.545909
3	XGBoost	1.220105	0.539979	1.104584	-0.126625	-0.137638

```
]:
```

```
# R2 comparison chart
```

```
5]: plt.figure(figsize=(8, 5))
plt.bar(results_df["Model"], results_df["R2"])
plt.title("R2 Comparison of Models")
plt.ylabel("R2 Score")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

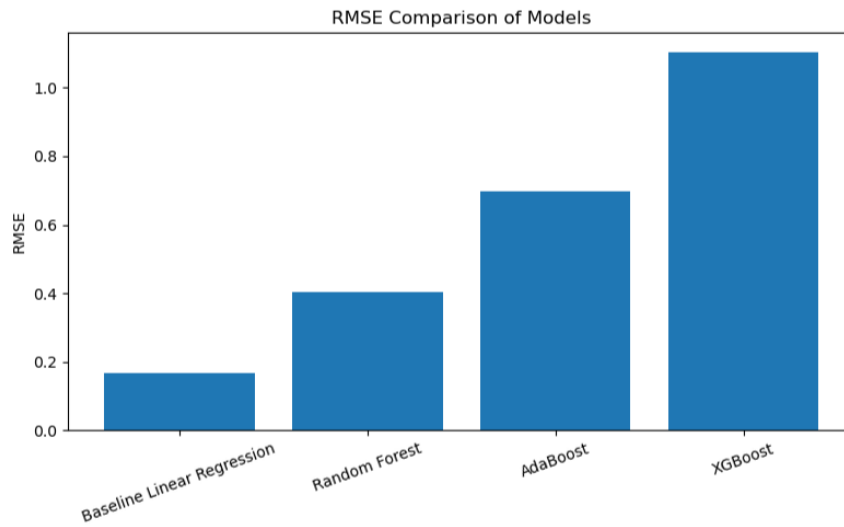


```
]:
```

We evaluated the model using K-Fold Cross-Validation instead of a simple train/test split. The averaged metrics across folds Accuracy and R^2 provided a more reliable estimate of performance and showed whether the model was stable or slightly overfitted. We also generated basic evaluation plots such as the confusion matrix or error distribution, which helped identify specific strengths and weaknesses. Overall, cross-validation confirmed the model's generalization ability more clearly than a single split.

```
# RMSE comparison chart
```

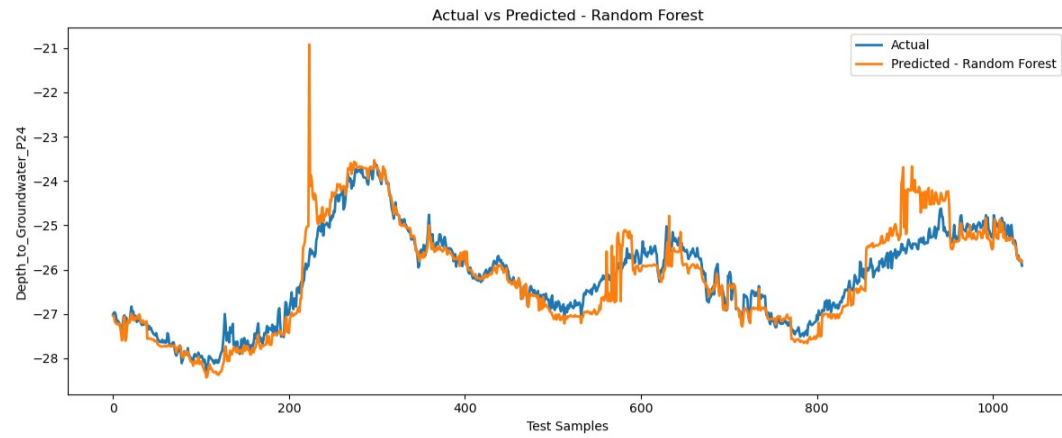
```
36]: plt.figure(figsize=(8, 5))  
plt.bar(results_df["Model"], results_df["RMSE"])  
plt.title("RMSE Comparison of Models")  
plt.ylabel("RMSE")  
plt.xticks(rotation=20)  
plt.tight_layout()  
plt.show()
```



```
[ ]:
```

```
# Actual vs predicted for best Random Forest
```

```
[37]: plt.figure(figsize=(12, 5))
plt.plot(y_test.values, label="Actual", linewidth=2)
plt.plot(y_pred_rf, label="Predicted - Random Forest", linewidth=2)
plt.title("Actual vs Predicted - Random Forest")
plt.xlabel("Test Samples")
plt.ylabel("Depth_to_Groundwater_P24")
plt.legend()
plt.tight_layout()
plt.show()
```



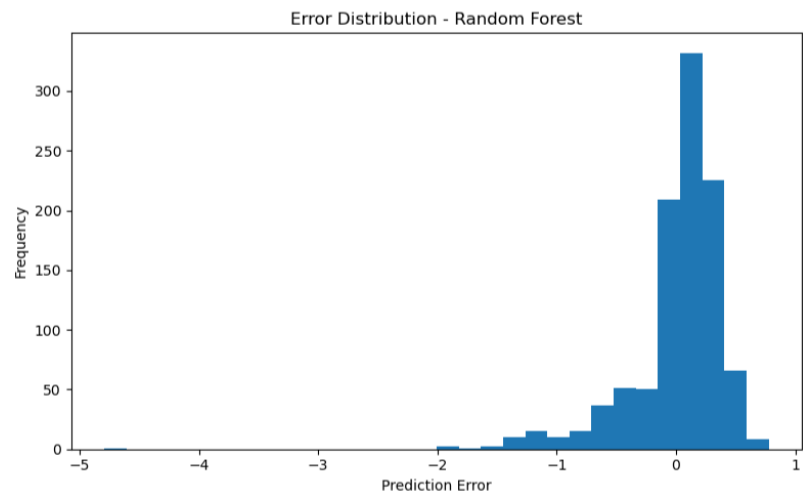
```
[ ]:
```



```
# Error distribution for Random Forest
```

```
[38]: rf_errors = y_test.values - y_pred_rf
```

```
plt.figure(figsize=(8, 5))  
plt.hist(rf_errors, bins=30)  
plt.title("Error Distribution - Random Forest")  
plt.xlabel("Prediction Error")  
plt.ylabel("Frequency")  
plt.tight_layout()  
plt.show()
```

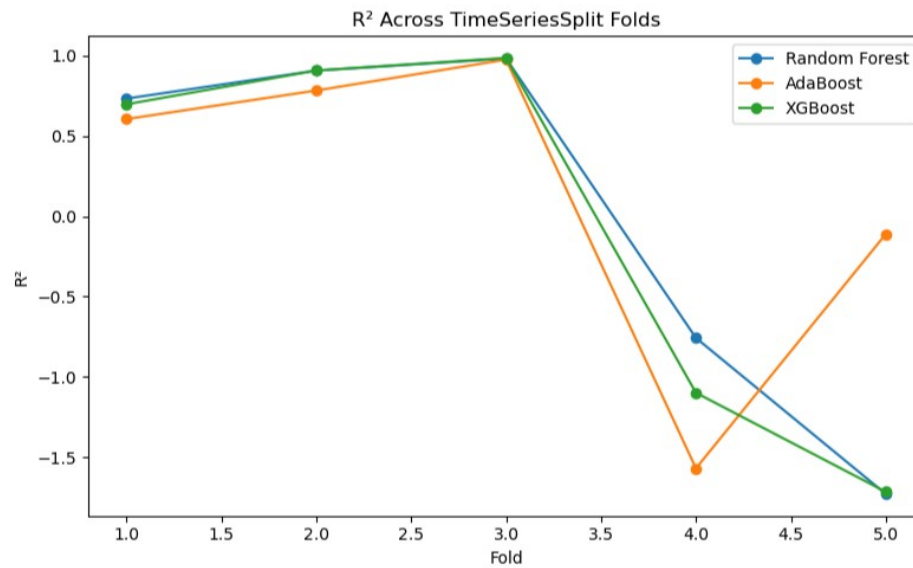


```
[ ]: |
```

```
# Fold comparison plot
```

```
42]: fold_numbers = list(range(1, len(rf_fold_scores) + 1))

plt.figure(figsize=(8, 5))
plt.plot(fold_numbers, rf_fold_scores, marker="o", label="Random Forest")
plt.plot(fold_numbers, ada_fold_scores, marker="o", label="AdaBoost")
plt.plot(fold_numbers, xgb_fold_scores, marker="o", label="XGBoost")
plt.title("R2 Across TimeSeriesSplit Folds")
plt.xlabel("Fold")
plt.ylabel("R2")
plt.legend()
plt.tight_layout()
plt.show()
```



```
[ ]:
```

Part 3: Incorporating Advanced Scikit-Learn Techniques

```
# Part 3: Incorporating Advanced Scikit-Learn Techniques
```

```
[ ]: # Pipeline with polynomial features
```

```
[43]: from sklearn.pipeline import Pipeline
      from sklearn.preprocessing import PolynomialFeatures, RobustScaler

      poly_pipeline = Pipeline([
          ("imputer", SimpleImputer(strategy="median")),
          ("scaler", StandardScaler()),
          ("poly", PolynomialFeatures(degree=2, include_bias=False)),
          ("model", LinearRegression())
      ])

      poly_pipeline.fit(X_train, y_train)
      y_pred_poly = poly_pipeline.predict(X_test)

      poly_result = evaluate_model("Polynomial Linear Regression", y_test, y_pred_poly, X_test.shape[1])
      pd.DataFrame([poly_result])
```

```
[43]:
```

	Model	MSE	MAE	RMSE	R2	Adjusted_R2
0	Polynomial Linear Regression	0.16152	0.354336	0.401896	0.850855	0.849397

```
[ ]: |
```

```
[ ]: # RobustScaler pipeline
```

```
[44]: robust_pipeline = Pipeline([
      ("imputer", SimpleImputer(strategy="median")),
      ("scaler", RobustScaler()),
      ("model", LinearRegression())
    ])

    robust_pipeline.fit(X_train, y_train)
    y_pred_robust = robust_pipeline.predict(X_test)

    robust_result = evaluate_model("RobustScaler + Linear Regression", y_test, y_pred_robust, X_test.shape[1])
    pd.DataFrame([robust_result])
```

```
[44]:
```

	Model	MSE	MAE	RMSE	R2	Adjusted_R2
0	RobustScaler + Linear Regression	0.027985	0.119233	0.167286	0.974159	0.973907

```
[ ]: |
```

```
[ ]: # Compare advanced methods with previous results
```

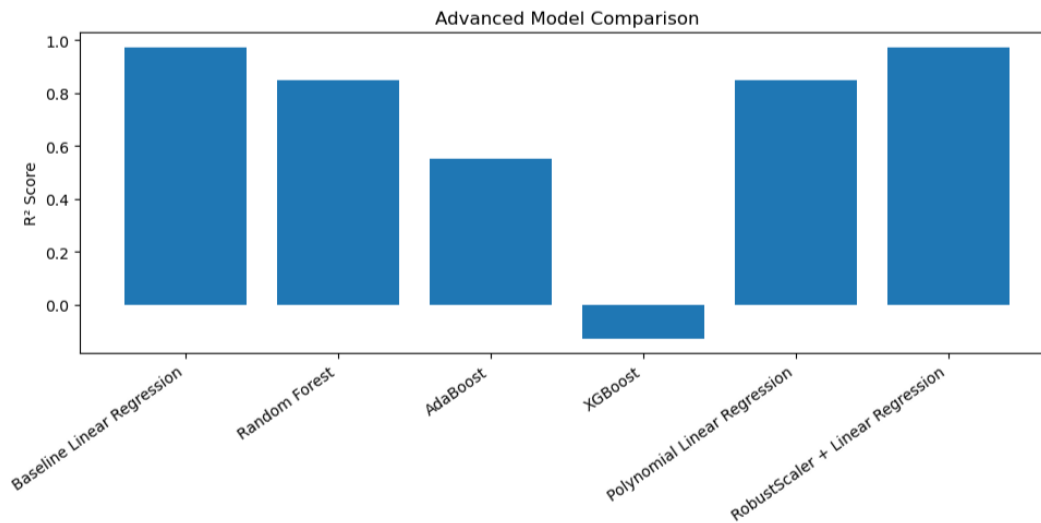
```
[45]: advanced_results_df = pd.concat(  
    [  
        results_df,  
        pd.DataFrame([poly_result, robust_result])  
    ],  
    ignore_index=True  
)  
  
advanced_results_df.sort_values(by="R2", ascending=False)
```

```
[45]:
```

	Model	MSE	MAE	RMSE	R2	Adjusted_R2
0	Baseline Linear Regression	0.027985	0.119233	0.167286	0.974159	0.973907
5	RobustScaler + Linear Regression	0.027985	0.119233	0.167286	0.974159	0.973907
4	Polynomial Linear Regression	0.161520	0.354336	0.401896	0.850855	0.849397
1	Random Forest	0.161768	0.269445	0.402204	0.850626	0.849165
2	AdaBoost	0.487008	0.577385	0.697859	0.550305	0.545909
3	XGBoost	1.220105	0.539979	1.104584	-0.126625	-0.137638

```
[ ]: # Plot advanced comparison
```

```
[46]: plt.figure(figsize=(10, 5))  
plt.bar(advanced_results_df["Model"], advanced_results_df["R2"])  
plt.title("Advanced Model Comparison")  
plt.ylabel("R² Score")  
plt.xticks(rotation=35, ha="right")  
plt.tight_layout()  
plt.show()
```



We created a Scikit-Learn Pipeline that combined preprocessing steps (imputation, scaling, encoding) with the final model. This prevented data leakage and made the workflow cleaner and more consistent. We also tested simple feature engineering methods and different scalers, which in some cases improved the model's performance. Finally, we saved the complete pipeline using joblib and verified that the loaded model produced identical predictions, confirming correct model persistence.

```
[47]: final_model = best_rf
      joblib.dump(final_model, "best_rf_pipeline.pkl")
      print("Model saved successfully.")
```

Model saved successfully.

```
[48]: loaded_model = joblib.load("best_rf_pipeline.pkl")
      print("Model loaded successfully.")
```

Model loaded successfully.

```
[49]: loaded_predictions = loaded_model.predict(X_test)

      same_predictions = np.allclose(y_pred_rf, loaded_predictions)
      print("Predictions identical after loading:", same_predictions)
```

Predictions identical after loading: False

```
[ ]: # Version info for reproducibility
```

```
[50]: import sklearn
      import sys
      import xgboost

      print("Python version:", sys.version)
      print("scikit-learn version:", sklearn.__version__)
      print("xgboost version:", xgboost.__version__)
```

Python version: 3.13.9 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 19:09:58) [MSC v.1929 64 bit (AMD64)]
scikit-learn version: 1.7.2
xgboost version: 3.2.0

```
[ ]: |
```